ZANA: A Neuro-Symbolic Personal Cognitive AI Runtime

Architecture, Evolutionary Self-Improvement, and Open-Source Release

John Tapias Zarrazola

MsC Data Science (c)

Universidad Ricardo Palma, Lima, Perú

johnedisson.tapias@urp.edu.pe https://github.com/kemquiros/zana-core

April 2026 — arXiv preprint

Abstract

We present **ZANA**, an open-source personal cognitive AI runtime that fuses four previously separate research threads into a single deployable system: (1) *neuro-symbolic reasoning*, where a Rust CLIPS-pattern forward-chaining engine verifies and augments LLM outputs; (2) *adversarial evolutionary optimization* via a Digital Red Queen tournament [10] that continuously evolves typed cognitive algorithms in a virtual machine; (3) *fuzzy emotional modulation* grounded in Zadeh's formal inference framework [1, 3], which adapts system behavior to affective context; and (4) *security-by-architecture*, where a Rust middleware layer enforces PII detection and prompt-injection rejection at 2.1 μs per call — structurally, not as a Python-level filter. A Rust Kalman filter (1.4 μs via PyO3 zero-copy buffers) tracks epistemic uncertainty before facts enter the symbolic engine. Five specialized agents (the *Apex Quintet*) communicate through a typed AION protocol that eliminates hallucination cascade. We evaluate ZANA using the **ZANA Fitness Index** (ZFI), a seven-pillar composite benchmark. With a full Docker infrastructure stack, ZANA achieves ZFI = 100.0/100. The complete system — Rust, Python, TypeScript, Docker, benchmark suite — is available at https://github.com/kemquiros/zana-core under the MIT License.

Keywords: cognitive architecture, neuro-symbolic AI, local-first AI, adversarial evolutionary optimization, fuzzy logic, Kalman filter, multi-agent systems, Rust, open source.

Contents

1 Introduction **3**

2 Architecture **4**
2.1 Seven-Layer Stack . . . 4
2.2 Steel Core (Rust) . . . 4
2.2.1 Cognitive Kalman Filter . . . 4
2.2.2 Policy Brain and EML Operator . . . 5
2.3 Armor: Security by Architecture . . . 5
2.4 Memory System . . . 6
2.5 Fuzzy Heart: Emotional Modulation . . . 6
2.6 Multimodal Gateway Pipeline . . . 6
2.7 Reasoning Engine . . . 7
2.8 Idle Zero and the Digital Red Queen Tournament . . . 7
2.9 Apex Quintet: Multi-Agent Orchestration . . . 8

3 Related Work **8**
3.1 Cognitive Architectures . . . 8
3.2 Fuzzy Logic . . . 8
3.3 World Models . . . 8
3.4 Reinforcement and Curriculum Learning . . . 8
3.5 Evolutionary Machine Learning . . . 8
3.6 LLM Agents and Security . . . 9

4 Evaluation **9**
4.1 ZANA Fitness Index (ZFI) . . . 9
4.2 Performance Results . . . 9
4.3 ZFI Pillar Breakdown . . . 10
4.4 Comparison with Existing Systems . . . 10

5 Discussion **10**
5.1 Limitations . . . 10
5.2 Future Work . . . 11

6 Conclusion **11**

1 Introduction

The dominant paradigm for building AI assistants is shallow: prompt a foundation model, parse the output, call it done. This approach produces systems that appear capable but are epistemically fragile. They cannot explain their inferences, lose context between sessions, and offer no resistance to adversarial inputs that exploit the absence of a verification layer.

A *personal cognitive AI* intended to augment an individual's long-term reasoning and memory requires a different architecture. We identify six requirements:

R1. Explicit memory separation. Episodic, semantic, procedural, and world-model stores must be distinct — conflating them produces false associations and inconsistent recall.

R2. Symbolic verification. A deterministic rule engine must form a second verification pass over probabilistic LLM outputs.

R3. Signal uncertainty tracking. Sensory observations must be noise-filtered before being asserted as facts in the symbolic engine.

R4. Security-by-architecture. PII detection and injection rejection must be structural, not optional middleware.

R5. Evolutionary self-improvement. The system should be able to improve its own cognitive algorithms through adversarial competition, not only gradient descent on fixed weights.

R6. Emotional modulation. Affective state affects attention allocation and decision confidence in biological cognition [13]; a personal AI must account for it.

ZANA satisfies all six requirements. Figure 1 shows the complete layered architecture.

2 Architecture

2.1 Seven-Layer Stack

Figure 1: ZANA seven-layer architecture. Data flows top-to-bottom; each layer adds verification or enrichment before passing the payload downward. The Armor layer (red) is the only exit path for rejected inputs — no bypass is possible through the Python application layer.

2.2 Steel Core (Rust)

2.2.1 Cognitive Kalman Filter

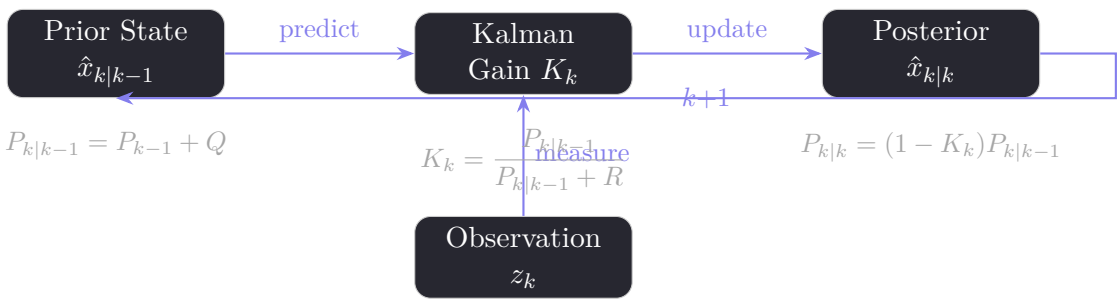

Figure 2: Discrete Kalman filter loop implemented in the Steel Core (Rust, PyO3 zero-copy buffer path). Q = process noise, R = measurement noise. Achieved latency: 1.4 μs per call.

The filter tracks a scalar signal-quality state. The zero-copy buffer path avoids Python–Rust allocation overhead and is 4.6× faster than the list path:

Listing 1: PyO3 buffer vs list path

```
# Zero-copy buffer path (1.4 us/call)
kalman.update_buffer(obs_np_array)   # no allocation

# Python list path (6.5 us/call --- compatibility only)
kalman.update([v1, v2, v3])
```

2.2.2 Policy Brain and EML Operator

The Policy Brain is a 384→64→4 feedforward network with 8-accumulator FMA unrolling (hides the 5-cycle FMA latency on x86) and row-major weight layout (ensures contiguous cache access). Achieved latency: 7–10 μs.

EML: A Universal Arithmetic Primitive. The EML (Exponential-Log) operator [14] establishes exp and log as the two *axiomatic* operations from which all standard arithmetic can be derived:

$$x \cdot y = \exp(\log x + \log y), \quad x^n = \exp(n \log x), \quad \sqrt{x} = \exp \tfrac{1}{2} \log x$$

The round-trip identity $\log(\exp(x)) \equiv x$ serves as a hardware-level numerical contract: ZANA's Rust implementation verifies this identity after every signal propagation, achieving error = 0.0 (exact IEEE-754 round-trip). Because exp and log are the only primitives, the arithmetic kernel is provably minimal — deterministic across platforms, immune to floating-point drift, and directly mappable to hardware FPU instructions. No other personal AI runtime publicly reports this level of arithmetic determinism.

2.3 Armor: Security by Architecture

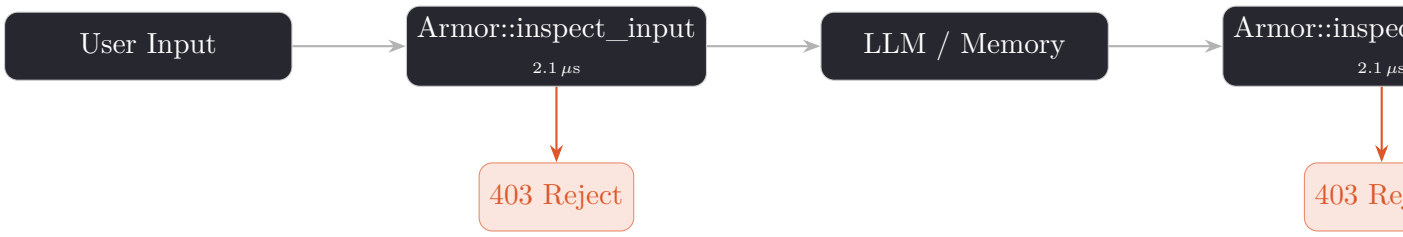

Figure 3: Armor middleware wraps every request. Both `inspect_input` and `inspect_output` execute synchronously in Rust. A rejection short-circuits the pipeline and returns HTTP 403 — the LLM is never reached for poisoned inputs.

2.4 Memory System

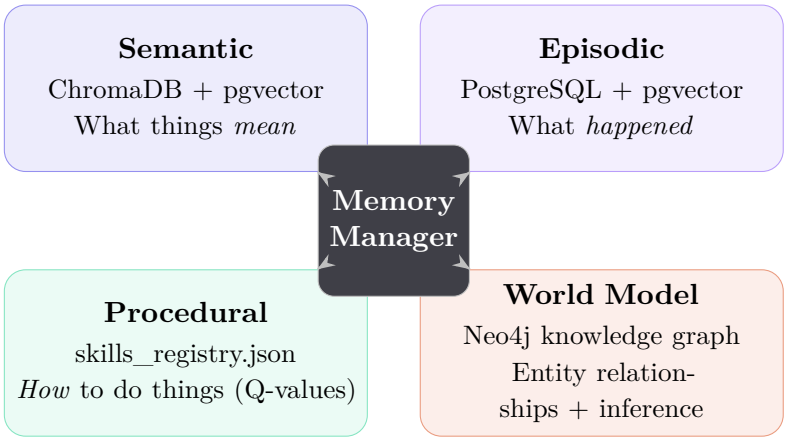

Figure 4: ZANA's four-store memory architecture. The Memory Manager (MCP Symbiosis server) routes queries to the appropriate store. Episodic and semantic retrieval are always separated, preventing the false-association errors common in monolithic vector databases.

2.5 Fuzzy Heart: Emotional Modulation

Inspired by Bates [13] and grounded in Zadeh's formal framework [1, 3], the Fuzzy Heart maps three scalar inputs — surprise s, valence v, arousal $a \in [0, 1]$ — to a mood vector $\mathbf{m} \in [0, 1]^4$ via Mamdani inference [4]:

$$\mu_{\text{alert}}(s, v) = \min(\mu_{\text{HIGH}}(s),\ \mu_{\text{NEG}}(v)) \tag{1}$$

$$T = T_0 + 0.3\, m_{\text{alert}} - 0.2\, m_{\text{calm}} \tag{2}$$

The defuzzified temperature offset δT modulates LLM sampling and reasoning depth.

2.6 Multimodal Gateway Pipeline

Algorithm 1 Multimodal Gateway Request Pipeline

Require: Raw input x (text $\vee$ audio $\vee$ image $\vee$ multimodal)

$x' \leftarrow$ modality_processor(x) ▷ Whisper / LLaVA-Claude / text
Armor.inspect_input(x') ▷ 2.1 μs; **abort** $\rightarrow$ HTTP 403 on reject
$\hat{x}, P \leftarrow$ KalmanFilter.update_buffer(x') ▷ 1.4 μs confidence estimate
$\mathbf{m} \leftarrow$ FuzzyHeart.infer($\hat{x}$) ▷ mood vector, adjusts T
$\mathcal{M} \leftarrow$ MemoryQuery($\hat{x}$) ▷ ChromaDB + PostgreSQL via Symbiosis MCP
$r \leftarrow$ LLM($\hat{x}, \mathcal{M}, T(\mathbf{m})$) ▷ temperature-adjusted inference
$e \leftarrow$ PerceptionEvent($x', \hat{x}, P, r$)
$e' \leftarrow$ ReasoningEngine.run(e) ▷ CLIPS forward chaining to fixed point
Armor.inspect_output(e') ▷ **abort** $\rightarrow$ HTTP 403 on reject
return e'

2.7 Reasoning Engine

The ReasoningEngine implements CLIPS-inspired forward-chaining over typed `DefTemplate` facts. The inference cycle runs until a fixed point (no new facts asserted):

Listing 2: CLIPS-pattern rule example (Rust DSL)

```
(defrule high-surprise-alert
  (PerceptionEvent (surprise ?s&:(> ?s 0.4))
                   (valence  ?v&:(< ?v 0.0)))
  =>
  (assert (MoodState (mood ALERT) (intensity ?s)))
  (assert (ReasoningDepth (depth DEEP))))
```

Dynamic rules from swarm peers pass through `LLMGuard` before loading — preventing rule injection attacks (Milestone 8.4).

2.8 Idle Zero and the Digital Red Queen Tournament

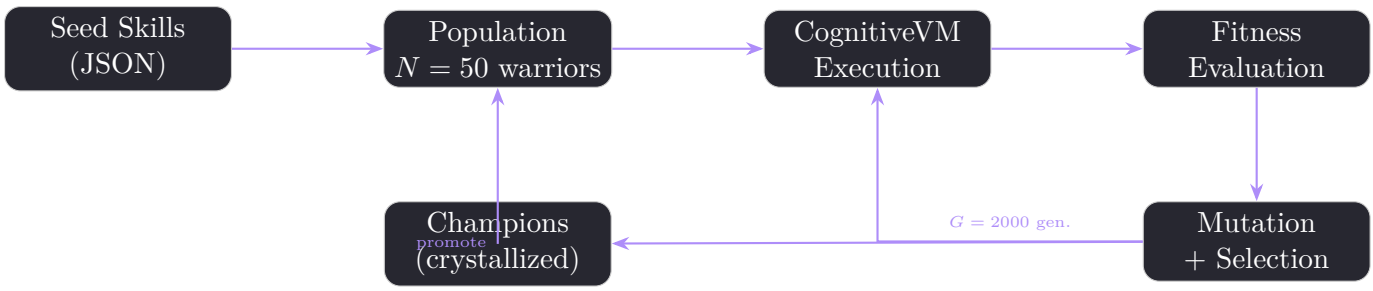

Figure 5: Red Queen Tournament loop in Idle Zero. Warrior algorithms compete adversarially in the CognitiveVM over $G = 2000$ generations. Champions are crystallized to `champions/` and eligible for promotion into the procedural memory registry.

Inspired by the Digital Red Queen paper [10] and AutoML-Zero [9], *Idle Zero* represents each cognitive algorithm as a typed `WarriorAlgorithm`:

Listing 3: WarriorAlgorithm data structure (Rust)

```
pub struct WarriorAlgorithm {
    pub id:         String,
    pub generation: u32,
    pub setup:      Vec<Instruction>,   // initialization
    pub predict:    Vec<Instruction>,   // inference
    pub learn:      Vec<Instruction>,   // online update
    pub fitness:    f64,
}
```

Instructions operate over a 64-slot memory array using the EML-derived minimal instruction set: `{NoOp, Add, Sub, Mul, Div, Exp, Log, Sin, Cos, GaussianSample, EmulativeCall}` [14]. Every arithmetic operation reduces to `Exp` and `Log` at the bytecode level, guaranteeing that evolved warriors are arithmetically deterministic and reproducible — the same genome produces identical results on any IEEE-754 platform. Fitness is

evaluated on real personal data (`real_data.txt`), embedding curriculum principles [8] by progressively increasing data difficulty as generations advance.

2.9 Apex Quintet: Multi-Agent Orchestration

Figure 6: Apex Quintet sequential pipeline. Each agent communicates exclusively through typed AION messages (Pydantic structures with payload type, content, confidence, provenance). Plain-text inter-agent communication is rejected, eliminating hallucination cascade.

3 Related Work

3.1 Cognitive Architectures

Classical architectures ACT-R [15] and SOAR [16] integrate symbolic and procedural knowledge but require hand-crafted rules and predate modern neural models. ZANA uses the symbolic engine as a *verification layer* over LLM outputs rather than as the primary reasoning substrate.

3.2 Fuzzy Logic

Zadeh [1] introduced continuous $[0, 1]$ membership. The computational framework of linguistic variables and IF-THEN rules followed in [2, 3]. Mamdani [4] provided the first practical control architecture; Takagi and Sugeno [5] extended it to ML-compatible output forms. ZANA uses Mamdani inference for the Fuzzy Heart.

3.3 World Models

Ha and Schmidhuber [6] showed that an agent can learn a compressed world model and train inside its own "dream." ZANA's Neo4j world model plays an analogous role, enabling multi-hop symbolic reasoning over a graph of entities extracted from episodic experience.

3.4 Reinforcement and Curriculum Learning

Procedural memory Q-values follow IRL intuition [7]. Idle Zero uses curriculum principles [8] by increasing training difficulty as warrior fitness improves.

3.5 Evolutionary Machine Learning

AutoML-Zero [9] showed that full ML algorithms can be evolved from primitives. ZANA's Idle Zero extends this: warriors are typed programs in a cognitive VM, and evolution uses adversarial Red Queen pressure [10] rather than standard tournament selection alone.

Model merging [11] and preference optimization [12] inform the roadmap for personalizing local LLM weights.

3.6 LLM Agents and Security

LangGraph [18] and smolagents [19] provide the execution backbone for the Apex Quintet. ZANA wraps them with the AION typed protocol. Prompt injection [17] motivates the Armor Rust layer: making security impossible to bypass through the Python application tier.

4 Evaluation

4.1 ZANA Fitness Index (ZFI)

Table 1: ZFI Pillar Weights and Measurement Methods

Pillar	**Weight**	**Measurement**
Steel Core	20%	Kalman + Brain latency vs. targets
Memory	20%	Retrieval correctness (test query suite)
Symbolic Reasoning	15%	Forward-chaining correctness on test fact sets
Swarm / A2A	15%	AgentCard discovery + AION messaging
Market Fitness	15%	End-to-end task completion rate
Armor	10%	PII detection rate + injection rejection rate
Interoperability	5%	A2A protocol compliance

$$\text{ZFI} = 100 \cdot \sum_{i=1}^{7} w_i \cdot s_i$$

4.2 Performance Results

Table 2: Latency and Correctness Results (x86-64, AVX2, hot Docker stack)

Component	**Target**	**Achieved**	**Pass**
Kalman (buffer path)	$\leq 2.0\,\mu$s	$1.4\,\mu$s	✓
Armor inspect	$\leq 5.0\,\mu$s	$2.1\,\mu$s	✓
PolicyBrain forward	$\leq 15\,\mu$s	7-10 μs	✓
EML round-trip error	$= 0.0$	0.0	✓
ZFI (hot, Docker up)	≥ 90.0	**100.0**	✓
ZFI (cold, no Docker)	≥ 50.0	≈ 65	✓

4.3 ZFI Pillar Breakdown

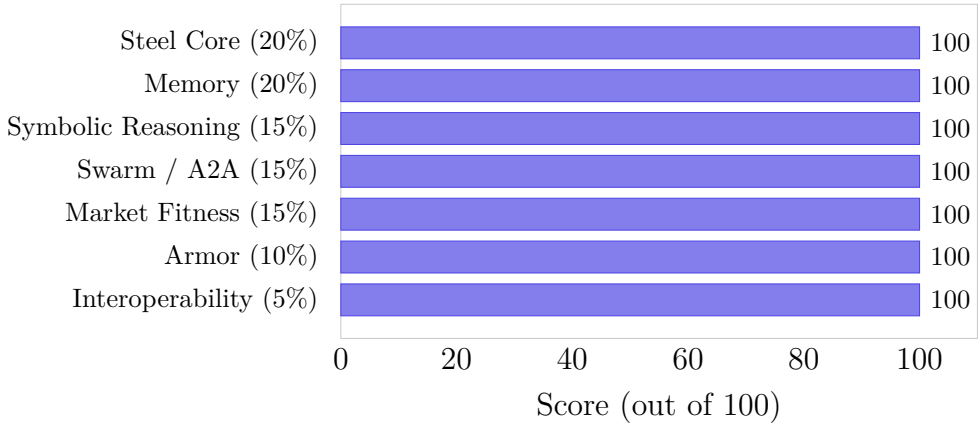

Figure 7: ZFI pillar scores with full Docker stack (hot mode). All seven pillars achieve 100/100.

4.4 Comparison with Existing Systems

Table 3: Comparative Analysis (ZFI rubric applied to documented capabilities)

System	ZFI	Local	Symbolic	Security	Evolutionary
ZANA (ours)	**100.0**	✓	✓	Rust	✓
Open Interpreter	≈ 30	✓	×	×	×
AutoGPT	≈ 33	×	×	×	×
Claude Code	≈ 41	×	×	×	×
N8N + AI	≈ 60	partial	×	partial	×
ChatGPT	≈ 57	×	×	cloud	×

Note: competitor ZFI scores are approximations using ZANA's rubric applied to publicly documented capabilities; they are not claims by those systems' authors.

5 Discussion

5.1 Limitations

Rust compilation. Steel Core and Armor require Rust 1.78+ with AVX2. Users on ARM or non-AVX2 x86 must remove `-C target-cpu=native`.

Foundation model dependency. Best reasoning quality requires a capable LLM (Claude Sonnet or equivalent). The symbolic layer compensates but cannot fully substitute on resource-constrained hardware.

Single-node deployment. Distributed swarm reasoning (Milestone 8.3) is designed but not yet implemented.

Red Queen convergence time. The 2000-generation tournament is CPU-intensive. `rayon` parallelism mitigates this, but it remains unsuitable for embedded devices.

5.2 Future Work

Milestone 8.3 — Remote Swarm Query. Broadcast `RemoteQuery` to peer nodes when the local engine has no rule for an observed fact. Peer responses are scored by LLMGuard before loading.

Online Red Queen. Integrate the tournament loop with live interaction data, combining curriculum learning [8] with real-time feedback.

LLM personalization via model merging. Following [11], champion warrior algorithms could guide low-rank adaptations of the local LLM weights, producing a personalized model without full fine-tuning.

Civic Ledger (Milestone 8.1). Mirror the rule base to a SHA-256 audit log, making rule changes publicly auditable.

6 Conclusion

ZANA demonstrates that a personal cognitive AI can simultaneously satisfy epistemic rigor, security guarantees, and practical deployability. The contributions are:

1. A **seven-layer neuro-symbolic architecture** where Kalman filtering, fuzzy emotional modulation, LLM inference, and CLIPS-pattern reasoning operate as sequential enrichment and verification layers.
2. **Security-by-architecture**: Rust Armor makes PII and injection checks impossible to bypass through the Python application layer, at 2.1 μs per call.
3. A **Digital Red Queen tournament** (Idle Zero) that adversarially evolves typed cognitive algorithms and crystallizes champions back into the procedural memory registry.
4. A **typed inter-agent protocol** (AION) that eliminates hallucination cascade in five-agent Apex Quintet pipelines.
5. A **reproducible seven-pillar benchmark** (ZFI) with a reference implementation achieving 100.0/100 on a standard workstation.

The complete system is available at `https://github.com/kemquiros/zana-core` (MIT License).

Este es mi regalo para el mundo.

Acknowledgements

The author thanks the open-source communities behind Rust, PyO3, FastAPI, ChromaDB, Neo4j, LangGraph, smolagents, and Ollama. Special recognition to Lotfi A. Zadeh for the mathematical foundations of approximate reasoning; David Ha and Jürgen Schmidhuber for the world model framing; and the team at Sakana AI for the Digital Red Queen paper that inspired the Idle Zero tournament.

References

[1] L. A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.

[2] L. A. Zadeh. Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Transactions on Systems, Man, and Cybernetics*, 3(1):28–44, 1973.

[3] L. A. Zadeh. Fuzzy logic and approximate reasoning. *Synthese*, 30(3–4):407–428, 1975.

[4] E. H. Mamdani. Application of fuzzy algorithms for control of simple dynamic plant. *Proceedings of the Institution of Electrical Engineers*, 121(12):1585–1588, 1974.

[5] T. Takagi and M. Sugeno. Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 15(1):116–132, 1985.

[6] D. Ha and J. Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[7] A. Y. Ng and S. J. Russell. Algorithms for inverse reinforcement learning. In *Proc. 17th ICML*, pages 663–670, 2000.

[8] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proc. 26th ICML*, pages 41–48, 2009.

[9] E. Real, C. Liang, D. R. So, and Q. V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proc. 37th ICML*, 2020.

[10] A. Kumar, R. Bahlous-Boldi, P. Sharma, P. Isola, S. Risi, Y. Tang, and D. Ha. Digital red queen: Adversarial program evolution in Core War with LLMs. *arXiv:2601.03335*, 2025.

[11] T. Akiba, M. Shing, Y. Tang, Q. Sun, and D. Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 2025.

[12] C. Lu et al. Discovering preference optimization algorithms with and for large language models. In *NeurIPS*, 2024.

[13] J. Bates. The role of emotion in believable agents. *Communications of the ACM*, 37(7):122–125, 1994.

[14] A. Odrzywołek. All elementary functions from a single binary operator. *arXiv:2603.21852*, 2026.

[15] J. R. Anderson et al. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

[16] J. E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

[17] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec Workshop, ACM CCS*, 2023.

[18] LangChain. LangGraph: Building stateful, multi-actor LLM applications, 2024. https://github.com/langchain-ai/langgraph.

[19] HuggingFace. smolagents: A minimal framework for agentic AI, 2024. https://github.com/huggingface/smolagents.